

Lecture 13. MemRef, Layout, Memory Space

DRAM/SRAM/Accumulator memory mapping for AI/NPU Compiler

Generated: 2026-05-07

1. 강의 목표

- MemRef type 을 shape/type/layout/memory-space 로 분해해 읽는다.
- strided layout 과 affine layout map 을 NPU address generation 으로 해석한다.
- DRAM/SRAM/ACCUM memory space 를 target ABI contract 로 설계한다.
- MatMul tile buffer 의 placement, footprint, descriptor 를 작성한다.
- layout/memory-space verifier 와 FileCheck test 를 설계한다.

2. 13 강의 위치

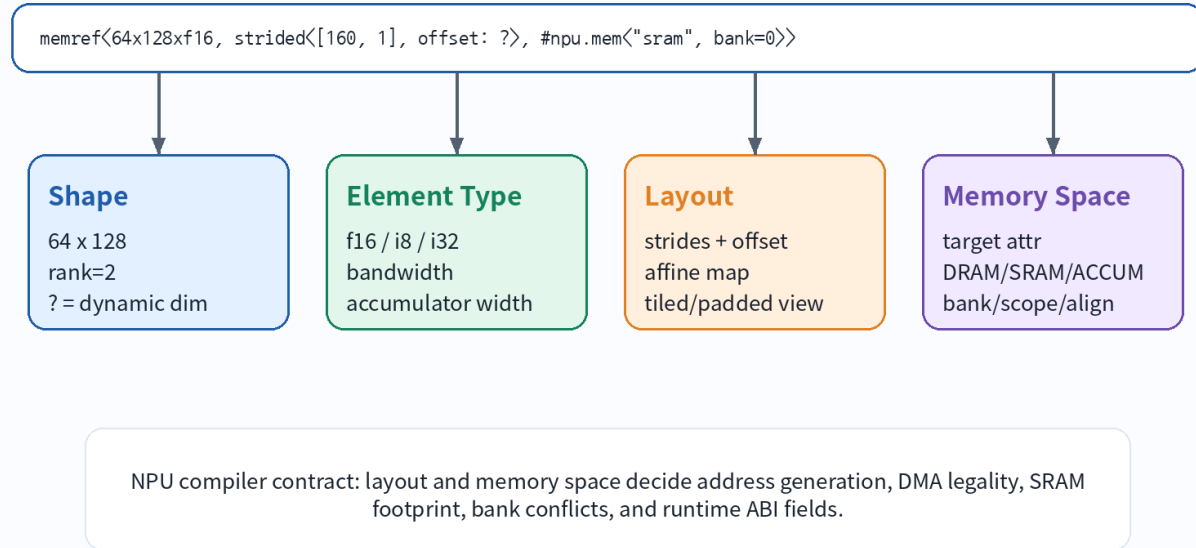
- 12 강에서 tensor-level value semantics 를 memref-level storage semantics 로 낮췄습니다. 13 강은 buffer 가 어디에 있고, 어떻게 주소 계산되며, 어떤 DMA/kernel ABI 로 전달되는지 결정하는 단계입니다.

3. MemRef Type Anatomy

- MemRef type 은 shape, element type, layout, memory space 를 포함합니다. NPU backend 는 이 정보를 kernel ABI 와 runtime descriptor 로 연결해야 합니다.

Pipeline Stage	IR 관점	NPU compiler decision
Before bufferization	tensor / linalg-on-tensors	fusion, tiling, layout 후보 결정
After bufferization	memref / view ops	allocation, alias, lifetime, memory space 명시
13 강 focus	memref<shape, layout, space>	DRAM/SRAM/ACCUM mapping, bank/layout/DMA descriptor

MemRef Type Anatomy



4. MemRef 는 pointer 가 아니다

일반적인 runtime descriptor 는 base pointer 외에도 offset, sizes, strides, dtype, memory space, alignment 정보를 포함합니다. Dynamic dimension 이나 dynamic stride 가 있으면 metadata 는 command buffer 로 내려가야 합니다.

Descriptor field	Why	NPU example
base/aligned pointer	storage 위치	DRAM physical address 또는 SRAM handle
offset	view 시작점	tile subview offset
sizes[]	dynamic shape	edge tile M/N 크기
strides[]	address calculation	padded row stride, banked layout
memory_space	target placement	DRAM=0, SRAM=1, ACCUM=2

5. Layout: strided form 과 affine map

```
#tile_16x32 = affine_map<(m, n) -> (m floordiv 16, n floordiv 32, m mod 16, n mod 32)>
memref<64x128xf16, #tile_16x32, 1>
memref<64x128xf16, strided<[160, 1], offset: 0>, 1>
```

Layout Map and Strided Metadata

Identity row-major

```
memref<4x8xf16>
strides = [8, 1]
offset = 0
```

$\text{addr} = \text{base} + (i*8+j)*2B$

Subview tile

```
memref<2x4xf16,
strided<[16,2], offset:9>>
```

view metadata; no copy

Tiled layout

```
#tile = affine_map<(m,n)->
(m div 16, n div 32,
m mod 16, n mod 32)>
```

outer tile + inner tile coordinates

Banked SRAM

```
stride padded to 64B
bank=(col/8) mod 4
```

reduce bank conflicts

```
linear_index = offset + sum_i(index_i * stride_i)
byte_address = base + linear_index * sizeof(element)
```

6. View operations: subview, transpose, extract_strided_metadata

Tile extraction 은 대부분 copy 가 아니라 metadata view 로 먼저 표현하는 것이 좋습니다. 이후 backend 가 DMA copy 또는 scalar copy 를 결정합니다.

```
%A_tile = memref.subview %A[%m0, %k0] [64, 128] [1, 1]
: memref<1024x1024xf16, strided<[1024,1], offset:0>, 0>
to memref<64x128xf16, strided<[1024,1], offset:?, 0>
```

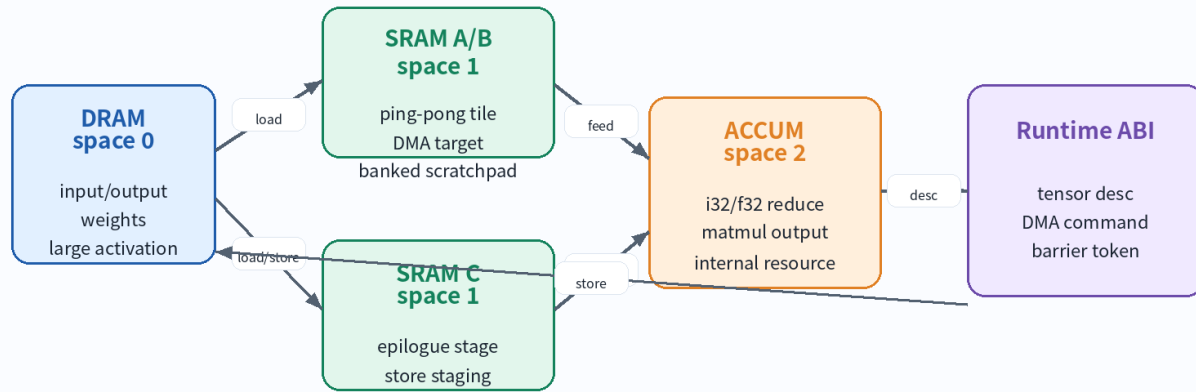
Operation	성격	NPU 사용처
memref.subview	metadata view	tile view, edge tile, DMA source view
memref.transpose	strided metadata permutation	layout-only transpose 판단
memref.extract_strided_metadata	base/offset/sizes/strides 추출	runtime descriptor lowering
memref.reinterpret_cast	metadata 재해석	descriptor rebuild, ABI bridge

7. Memory Space = target-specific ABI contract

```
memref<64x128xf16, 0> // DRAM
memref<64x128xf16, 1> // SRAM
memref<64x64xi32, 2> // ACCUM
memref<64x128xf16, strided<[160,1], offset:0>, #npv.mem<"sram", bank=0, align=64>>
```

Memory space	Hardware resource	Typical object	Verifier rule
DRAM	External memory	input/output activations, weights	DMA reachable, descriptor-visible, alignment valid
SRAM	Scratchpad	A/B/C tiles, ping-pong buffers	fits capacity, bank assignment valid, lifetime non-overlap
ACCUM	Accumulator	matmul/reduction output	legal producer/consumer, no illegal escape
CONST	Constant memory	weights, LUTs	read-only, prefetch policy consistent

DRAM / SRAM / ACCUM Mapping



Rule: memory_space is a placement + legality + descriptor contract, not only a numeric tag.

8. DRAM/SRAM/ACCUM MatMul mapping case study

$A[M \times K]$ in DRAM $\rightarrow A_tile[M_t \times K_t]$ in SRAM bank0
 $B[K \times N]$ in DRAM $\rightarrow B_tile[K_t \times N_t]$ in SRAM bank1
 $C_acc[M_t \times N_t]$ in ACCUM $\rightarrow C_stage[M_t \times N_t]$ in SRAM $\rightarrow C[M \times N]$ in DRAM

Buffer	Shape	Element	Space	Purpose
A_tile ping/pong	64x128	f16	SRAM bank0	activation tile load
B_tile ping/pong	128x64	f16	SRAM bank1	weight tile load
C_acc	64x64	i32	ACCUM	K reduction accumulate
C_stage	64x64	i8/f16	SRAM	requant/activation/store staging

9. Footprint 계산과 layout decision

$A_tile = M_t * K_t * \text{sizeof}(A)$
 $B_tile = K_t * N_t * \text{sizeof}(B)$
 $C_acc = M_t * N_t * \text{sizeof}(\text{accum})$
 $SRAM_total = \text{pingpong} * (A_tile + B_tile) + C_stage + \text{tags} + \text{padding}$

SRAM Bank / Padding Intuition

0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3
0	0	0	0	1	1	1	1	2	2	2	2	3	3	3

Compiler checks

- vector lane access pattern
- row stride alignment
- DMA burst size
- padding footprint
- bank assignment attr

Example: $\text{bank} = \text{floor}(\text{col} / 4) \bmod 4$. Padding row stride can shift the next row and reduce repeated same-bank pressure.

10. Runtime descriptor lowering

MemRef Metadata -> NPU Runtime Descriptor

MLIR memref

1. base/aligned pointer
2. offset
3. sizes[rank]
4. strides[rank]
5. element type
6. layout map
7. memory space
8. alignment

NPU descriptor

1. physical address or handle
2. byte offset
3. rank + dims
4. byte strides
5. dtype enum
6. layout id or raw map
7. space id + bank
8. alignment / flags

Verifier: type/layout/space must match kernel ABI and DMA legality

```
struct NpuTensorDesc {
    uint64_t handle_or_phys_addr;
    uint32_t byte_offset;
    uint16_t rank, dtype, memory_space, layout_id;
    int64_t sizes[MAX_RANK];
    int64_t byte_strides[MAX_RANK];
};
```

11. Pass design: memory-space assignment

Step	Input	Output/check
Classify buffers	memref.alloc, block args, subview	DRAM/SRAM/ACCUM/CONST 후보
Analyze lifetime	def-use, dominance, schedule region	reuse 가능성, ping-pong 필요성
Assign space	tile role, producer/consumer	memref type or npu.alloc op
Assign layout	kernel contract, vector lanes	strides, padding, layout id
Verify	capacity, alignment, escape	diagnostic or legal IR

one-shot-bufferize -> npu-classify-buffers -> npu-assign-memory-spaces -> npu-propagate-layout -> npu-insert-dma -> npu-lower-runtime-descriptors

12. Regression test checklist

- SRAM allocation 이 예상한 memory space 로 남는지 CHECK 한다.
- unexpected memref.copy 가 생기지 않는지 CHECK-NOT 으로 확인한다.
- unsupported layout 은 negative diagnostic 으로 잡는다.
- ACCUM memory space 가 function boundary 로 escape 하지 않는지 확인한다.
- dynamic offset/stride descriptor field 가 runtime lowering 에서 보존되는지 확인한다.

13. 실습 산출물

Exercise	Task	Output
A	memref type anatomy annotation	shape/type/layout/space 표
B	subview tile offset/stride 계산	tile view descriptor
C	MatMul tile memory mapping	DRAM/SRAM/ACCUM table
D	SRAM footprint 계산	capacity pass/fail decision
E	runtime descriptor 설계	C struct 또는 YAML

14. Takeaways

- MemRef 는 storage semantics 를 가진 structured reference 이며 pointer 하나가 아니다.
- Layout 은 address generation, DMA burst, bank conflict, vectorization 을 결정한다.
- Memory space 는 target-specific attribute 이므로 NPU backend 의 ABI contract 로 설계해야 한다.
- DRAM/SRAM/ACCUM mapping 은 correctness + performance + runtime ABI 문제다.

References

- MLIR MemRef Dialect: <https://mlir.llvm.org/docs/Dialects/MemRef/>
- MLIR Builtin Dialect - MemRef type: <https://mlir.llvm.org/docs/Dialects/Builtin/>
- MLIR Data Layout Modeling: <https://mlir.llvm.org/docs/DataLayout/>
- MLIR FAQ - MemRef is not only a pointer: https://mlir.llvm.org/getting_started/Faq/
- MLIR Passes: <https://mlir.llvm.org/docs/Passes/>